GhostWork

How it works, step by step, end to end

A walkthrough script you can read aloud or follow during a live demo. It follows one automation from sign-up to the finished result, and explains what the software does at each stage.

Written 30 October 2026, from the project source code.

How to use this script

Part 4 is the heart of it: 15 steps, each split three ways. **SAY** is what to tell your audience. **DO / SEE** is what to click and what appears. **BEHIND THE SCENES** is what the software is doing, for when someone asks how it works.

How to use this script 2

Part 1: What GhostWork is 3

Part 2: The pieces 3

Part 3: Starting everything 4

Part 4: The end-to-end walkthrough 5

Part 5: One run, in detail 8

Part 6: Security in plain terms 8

Part 7: Mobile and web stay the same 9

Part 8: How it is tested 9

Part 9: What is not finished 9

Part 1: What GhostWork is

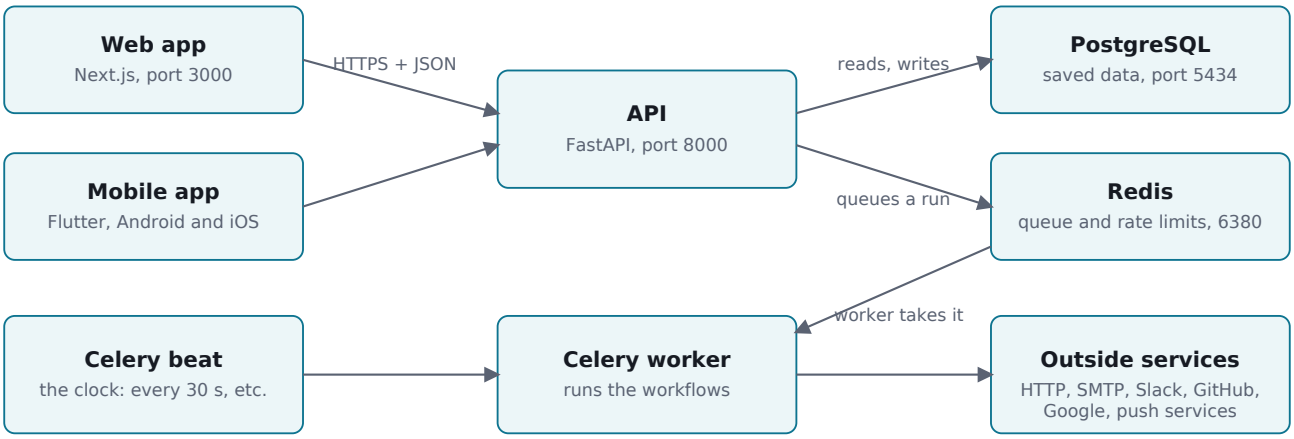
GhostWork is a workflow automation platform. You connect steps on a visual canvas, such as "when this web address is called, fetch some data, check it, then send an email", and GhostWork runs them for you with no code.

Its central idea is the **Living Workflow**: the same canvas you design on also shows what the automation is doing while it runs, and what each step did afterwards. If something fails, you see which step failed and why, in the place you built it.

The vocabulary

Word	Meaning
Workflow	One automation: a set of connected steps.
Trigger	What starts it: a manual click, a schedule (every 5 minutes or longer), or a call to its webhook address.
Step	One action: HTTP request, transform, condition, email, Slack message, GitHub issue, Google Sheets append or read.
Run (execution)	One time the workflow ran, with its own record of every step.
Integration	A saved, encrypted connection to an outside service.

Part 2: The pieces



The worker also writes every step result to PostgreSQL. That is what the apps read back.

Piece	Job
Web app (`frontend/`)	The screens in a browser. It holds no business rules beyond checking forms before they are sent.
Mobile app (`mobile/`)	The same screens natively in Flutter, talking to the same API.
API (`backend/app/api`)	The single source of truth. Every rule, permission and ownership check is enforced here.
Worker (`backend/app/workers`)	Runs workflows in the background, so a slow step never holds up the API.
Beat	A clock. Every 30 seconds it starts due schedules, every 2 minutes it cleans up stuck runs, every 6 hours it deletes old run data.

Piece	Job
PostgreSQL	Permanent storage: users, workflows, runs, steps, encrypted integrations.
Redis	The queue between API and worker, plus rate-limit counters.
Mailpit	A development-only inbox that catches emails, viewable on port 8026.

Part 3: Starting everything

You need Docker, Python, Node.js and, for mobile, Flutter. The first-time setup, including the `.env` files, is in the project README. After that, run each block below in its own terminal, starting from the project folder.

```
# Terminal 1: database, Redis and mail
docker compose up -d db redis mail

# Terminals 2, 3 and 4: the API, the worker and the clock (each from backend/)
.venv/bin/alembic upgrade head          # brings the database up to date; once is enough
.venv/bin/uvicorn app.main:app --reload --port 8000
.venv/bin/celery -A app.workers.celery_app worker --loglevel=info
.venv/bin/celery -A app.workers.celery_app beat --loglevel=info

# Terminal 5: the website (from frontend/)
npm run dev                             # then open http://localhost:3000
```

API documentation is generated automatically at `http://localhost:8000/docs`. On this machine, memory is tight: avoid running builds and browser tests at the same time.

Part 4: The end-to-end walkthrough

The example: **a workflow that checks whether a website is up, and reports the result.** It needs no outside accounts, so it works in any demo. This follows the "HTTP endpoint monitor" template's idea; steps 5 to 7 build it by hand so your audience sees the builder.

Step 1: Open the public site	
SAY	GhostWork automates repetitive work, and shows you exactly what it did.
DO / SEE	Open `localhost:3000`. Show the landing page, Features, Pricing and Docs.
BEHIND THE SCENES	These pages are static Next.js pages and need no login. Nothing here touches user data.

Step 2: Create an account	
SAY	Sign-up asks you to read and accept the Terms before an account exists.
DO / SEE	Go to Create account. Enter name, email and password, open the Terms, tick the box, submit. You land on the dashboard.
BEHIND THE SCENES	The API rejects sign-ups without acceptance, and records which Terms version was accepted and when. The password is stored only as an Argon2 hash. Sign-up attempts are rate-limited.

Step 3: Stay signed in safely	
SAY	Sessions are short-lived and can be cancelled from any device.
DO / SEE	Open Settings, Sessions, to show the signed-in devices.
BEHIND THE SCENES	The app holds a 15-minute access token in memory. A longer-lived refresh token sits in an HttpOnly cookie that page scripts can't read (the phone keychain or keystore on mobile). Each refresh replaces it, and reuse of an old token revokes the session as suspected theft.

Step 4: See the dashboard	
SAY	Everything on this page is computed from real runs. A new account really is empty.
DO / SEE	Point out the metrics, the 14-day trend and the health summary.
BEHIND THE SCENES	The API counts runs in PostgreSQL for this user only. While a run is in progress, the page refreshes every 3 seconds.

Step 5: Create a workflow	
SAY	Now we build the automation.
DO / SEE	Workflows, then create one and name it "Website check". The builder opens with the canvas, a step library and a settings panel.
BEHIND THE SCENES	The API saves it as a draft owned by your account. Every later request checks that ownership on the server, so nobody else can read or change it.

Step 6: Add and connect the steps	
SAY	Each box is one action. The lines decide what happens next.
DO / SEE	Add a Manual trigger, an HTTP request, and a Condition. Drag between handles to connect them. Then add one Transform to each of the condition's True and False outputs.
BEHIND THE SCENES	The canvas is stored as JSON: a list of nodes and a list of edges. Undo and redo keep a history in the browser until you save.

Step 7: Configure the steps	
SAY	Steps can use results from earlier steps.
DO / SEE	HTTP request: GET `https://example.com`. Condition: check that the HTTP step's status equals 200. Transforms: name them "Up" and "Down".
BEHIND THE SCENES	A reference such as `{{steps.check.status_code}}` is filled in at run time. It is a simple lookup, never executed as code, so users can't run code on the server.

Step 8: Save and validate	
SAY	GhostWork tells you what's wrong before anything runs.
DO / SEE	Press Ctrl+S. Leave a required field empty to show the error on that step, then fix it.
BEHIND THE SCENES	The API checks there is exactly one trigger, every step is reachable, there are no loops, required settings are filled, references resolve, and any integrations belong to you.

Step 9: Run it	
SAY	One click, and the canvas comes alive.
DO / SEE	Click Run. Watch each step change status on the canvas.
BEHIND THE SCENES	The API validates again, checks your limit on simultaneous runs, stores a run with a **snapshot** of the workflow, creates a pending record for each step, puts the job on the Redis queue, and replies "202 Accepted" immediately. The app then asks for the run's status once a second.

Step 10: The worker does the work	
SAY	The website is just watching. A separate process is doing the job.
DO / SEE	Keep watching the canvas: HTTP request, then Condition, then the matching Transform. The other branch shows as Skipped.
BEHIND THE SCENES	The worker marks the run as running and goes through the steps in dependency order, checking before each one whether you cancelled. A step runs only if an earlier step fired into it. Each step's status, attempts, output, error and timing are saved as it goes. See Part 5.

Step 11: Inspect the result	
SAY	Every run keeps a full record.
DO / SEE	Open Executions, then this run. Click a step to see its output or error. Try Re-run.
BEHIND THE SCENES	Because the run used a snapshot, editing the workflow later never alters this history. Run data is deleted after 90 days.

Step 12: See a failure	
SAY	The real test of an automation tool is what happens when something breaks.
DO / SEE	Change the URL to one that fails, and run it. The failing step is marked with an icon and text, not colour alone, and the error is shown. A notification appears.
BEHIND THE SCENES	The run is marked failed, remaining steps are skipped, and the owner is notified according to their preferences: in-app, email and push.

Step 13: Automate it	
SAY	Now it runs without you.
DO / SEE	Swap the trigger to Schedule (every 5 minutes or more) or Webhook. Click Activate and confirm.
BEHIND THE SCENES	**Schedule:** every 30 seconds beat finds due workflows and queues them. **Webhook:** the workflow gets a secret address, `/api/v1/hooks/<token>`. A call to it queues a run, with the request body available to the steps. Signatures are optional.

Step 14: Connect a service	
SAY	Real automations reach other tools.
DO / SEE	Integrations: connect Slack, email, GitHub or Google Sheets. Add the matching step to a workflow.
BEHIND THE SCENES	Credentials are encrypted with Fernet before being stored, and are never returned by the API or written to logs. GitHub uses a personal access token. Google uses a service account.

Step 15: Turn on push notifications	
SAY	You find out about failures even with the app closed.
DO / SEE	Settings, Notifications: turn on push and send a test. In the browser, a system notification appears.
BEHIND THE SCENES	The browser creates a push address, the API stores it, and on failure the API sends an encrypted message through the browser maker's push service. A service worker (`public/sw.js`) displays it. Signing out removes the device.

Part 5: One run, in detail



1. **Trigger.** A click (`POST /workflows/{id}/run`), a due schedule, or a webhook call. All three end in the same function, `queue_execution`.
2. **Validate.** The workflow must pass full validation, and you must be under the limit on simultaneous runs. Otherwise the request is refused with an explanation.
3. **Record.** A run row stores the trigger data and a copy of the definition, with one pending row per step.
4. **Queue.** The run's ID goes to Redis via Celery. The API's work is finished.
5. **Execute.** The worker locks the run and ignores it if it isn't "queued", so a duplicate delivery can't run it twice. It builds a context holding trigger data and step results, then works through the steps. Results become available to later steps as `steps.<id>`.
6. **Finish.** The run becomes succeeded, failed or cancelled, its duration is stored, and notifications go out. Push is best effort, so a push problem never changes the run's result.

When things go wrong

Situation	What happens
A step fails	The run is marked failed with the error. Later steps are skipped. HTTP steps can retry first.
You press Cancel	The worker sees it before the next step, and the run ends as cancelled.
A run takes too long	The worker's time limit ends it as failed, with a clear message.
A worker crashes	Every 2 minutes a clean-up task finds stuck runs and closes them.

Part 6: Security in plain terms

Risk	Protection
Stolen passwords	Argon2 hashing. Login takes a similar path for unknown accounts, so it doesn't reveal who has one.
Seeing someone else's data	Ownership is checked on the server on every request.
Stolen credentials	Integration secrets are encrypted at rest, and never returned or logged.
Workflows attacking internal systems (SSRF)	Outgoing requests are checked: private and internal addresses are refused where required, the connection is pinned to the checked IP, and redirects are re-checked. Size and time limits apply.
Forged requests (CSRF)	Cookie-based endpoints need a special header and an allowed origin.
Running malicious code	Users can't run code: steps come from a fixed list, and templates are lookups only.
Fake webhook calls	Secret addresses, optional HMAC-SHA256 signatures, size limits and rate limits.

Risk	Protection
Brute force	Rate limits on sign-up, login, refresh and password reset.

Part 7: Mobile and web stay the same

The Flutter app has the same features, screens and wording as the website, including the visual builder: drag to move, drag between handles to connect, pinch to zoom. Both use the same API, so a workflow built on one appears on the other. The Terms text is one shared file, and `scripts/sync-shared.sh` keeps the copies identical. Layouts for tablets and desktops differ from the web.

Part 8: How it is tested

Layer	What it checks
Backend (73 tests, pytest)	Runs against a real PostgreSQL test database: auth, workflows, execution, webhooks, security, integrations and push, including real Web Push encryption.
Browser (Playwright)	Drives the real stack in a browser like a user: sign-up, building, running, responsive layouts.
Mobile (28 tests)	Models, API client auth, builder logic, redirects, widgets, sign-in flow.
CI (GitHub Actions)	Runs backend, frontend, shared-content and mobile checks on each push and pull request.

Part 9: What is not finished

State these plainly if asked. They are the honest limits of the project today.

- **Not production-ready.** It has not been load-tested or independently security-tested. See the production checklist in the README.
- **GitHub and Google Sheets** are tested only against simulated responses, because no real credentials were available. Try them with a real account before relying on them.
- **Browser push** was verified for real, delivered through Google's push service in Chrome.
- **Android and iOS push** needs your own Firebase project and has never run on a real device.
- **The Terms and Privacy page** have not been reviewed by a lawyer. `docs/legal-review-brief.md` is prepared for that review.
- **Not built:** team workspaces, billing and sign-in with Google or GitHub.

One-minute summary

You design a workflow on a canvas. The API validates it and stores it. When a click, a clock or a webhook triggers it, the API saves a snapshot and queues a job through Redis. A background worker runs the steps in order and saves each result to PostgreSQL. The web and mobile apps read those results, so the same canvas you built on shows you, live, what happened. If it fails, you are told.